

Protocollo MQTT

Introdução

- MQTT significa MQ Telemetry Transport.
- É um protocolo de mensagens de publicação/assinatura, extremamente simples e leve, projetado para dispositivos restritos e redes de pequena largura de banda, alta latência ou não confiáveis.
- Os princípios de projeto são minimizar a largura de banda da rede e os requisitos de recursos do dispositivo, enquanto também tenta garantir a confiabilidade e algum grau de garantia de entrega.
- Esses princípios também o tornam o protocolo ideal para o emergente mundo de dispositivos conectados da "Internet das coisas" e também para aplicações móveis onde a largura de banda e a energia da bateria são requisitos importantes.

Introdução

- MQTT foi inventado pelo Dr. Andy Stanford-Clark da IBM, e Arlen Nipper da Arcom (agora Eurotech), em 1999.
- A especificação do protocolo foi publicada com uma licença “royalty-free” por muitos anos, e empresas como a Eurotech (anteriormente conhecida como Arcom) implementaram o protocolo em seus produtos.
- Em novembro de 2011, a IBM e a Eurotech anunciaram sua participação conjunta no Eclipse M2M Industry Working Group e doação do código MQTT para o projeto.
- Em 2016, MQTT foi considerado como um padrão ISO (ISO/IEC 20922) publicado pelo [OASIS](#).

Características

- O protocolo é executado sobre TCP/IP, ou sobre outros protocolos de rede que fornecem conexões bidirecionais, sem perdas, ordenadas. Suas características incluem:
 - Uso do padrão de mensagem de publicação/assinatura que fornece uma distribuição de mensagens de um-para-muitos além de desacoplamento de aplicações;
 - Um transporte de mensagens que é agnóstico para o conteúdo da mensagem enviada.
 - Uma pequena sobrecarga de transporte e trocas de protocolos minimizadas para reduzir o tráfego da rede.
 - Um mecanismo para notificar as partes interessadas quando ocorre uma desconexão anormal.

Características

- Existem três níveis de qualidade de serviço para entrega de mensagens:
 - "No máximo uma vez", onde as mensagens são entregues de acordo com os melhores esforços do ambiente operacional. Perda de mensagens podem ocorrer. Este nível poderia ser usado, por exemplo, com os dados de sensores de ambiente, onde não importa se uma leitura individual é perdida, já que uma nova leitura será publicada logo depois.
 - "Pelo menos uma vez", onde é assegurado que as mensagens vão chegar, mas duplicatas podem ocorrer.
 - "Exatamente uma vez", onde é assegurado que a mensagem irá chegar exatamente uma vez. Esse nível poderia ser usado, por exemplo, com sistemas de faturamento onde mensagens duplicadas ou perdidas poderiam levar à aplicação de uma tarifa incorreta.

Diagrama de Comunicação

MQTT COMMUNICATION DIAGRAM

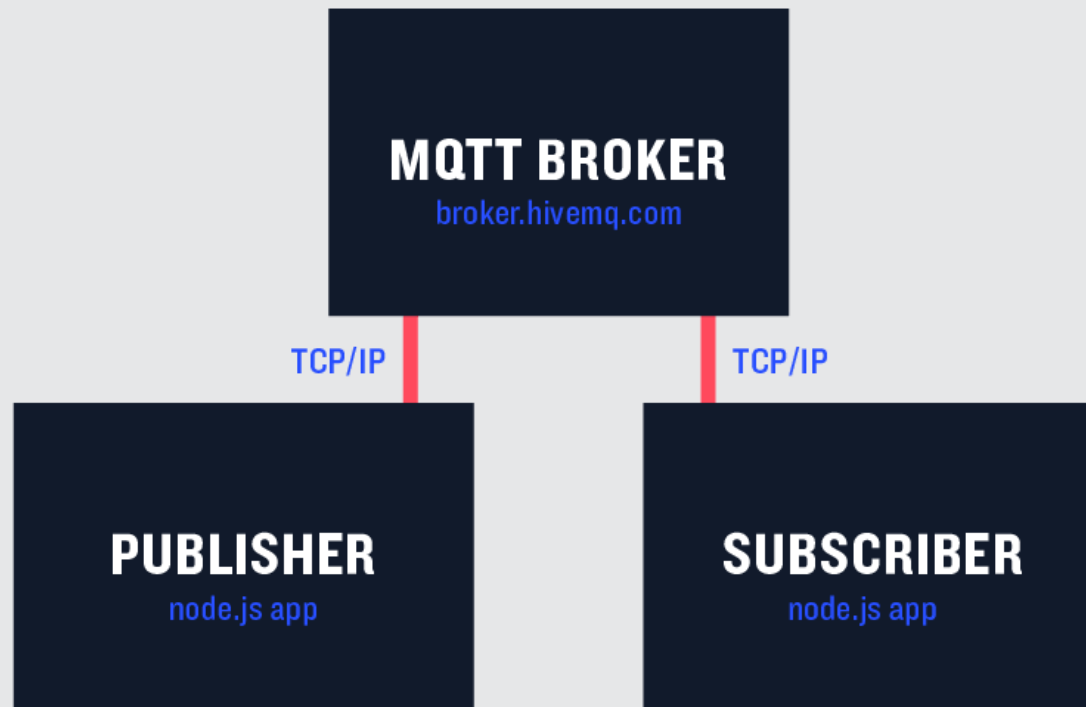


Diagrama de Comunicação

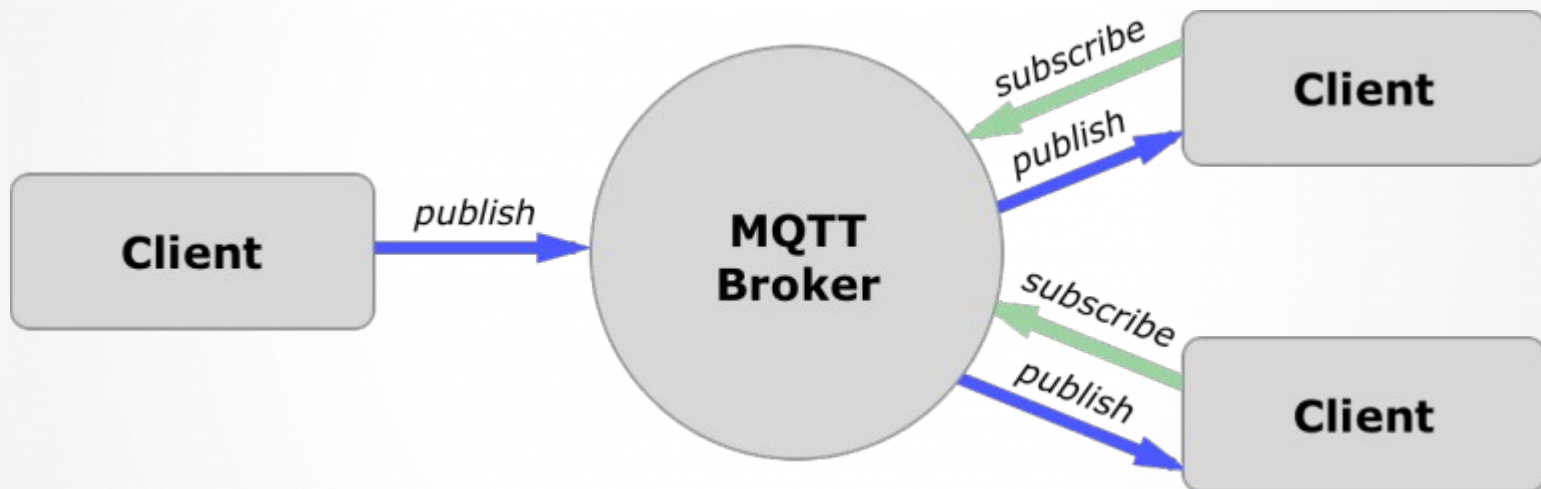
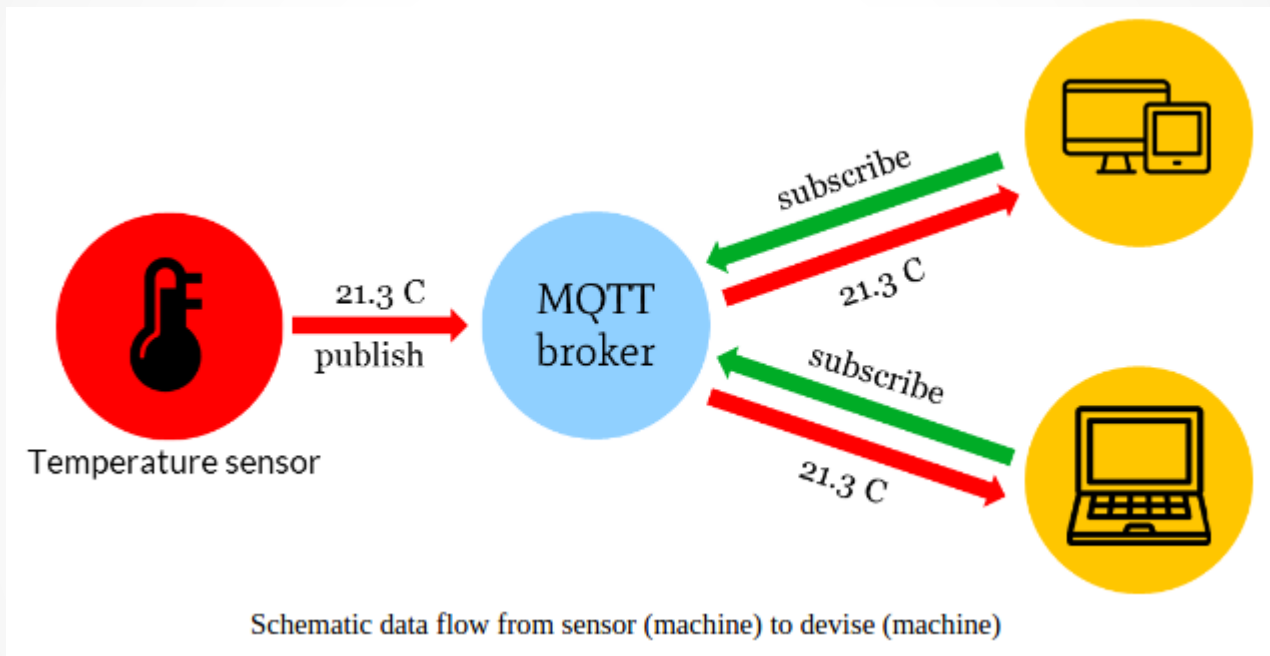


Diagrama de Comunicação



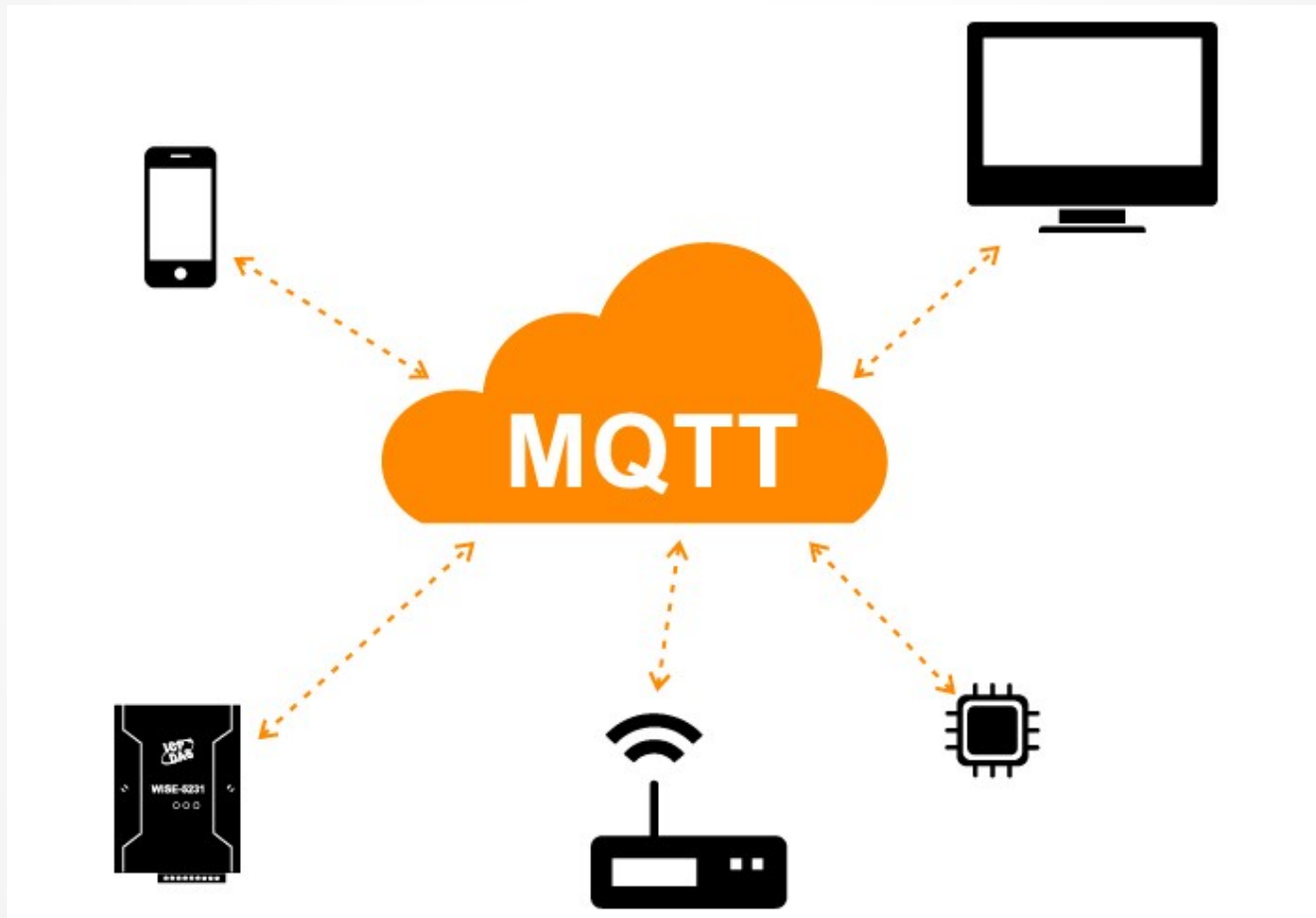
Por que não usar HTTP

- O protocolo HTTP é mais lento, tem mais overhead e consumo de energia que o protocolo MQTT.
- Mais lento porque ele utiliza pacotes de dados maiores para se comunicar com o servidor
- O protocolo HTTP abre e fecha a conexão a cada pedido, enquanto que o MQTT permanece ativo para manter os canais sempre abertos entre o servidor “broker” e os seus clientes.
- O consumo de energia é maior já que leva mais tempo e usa mais pacotes de dados.

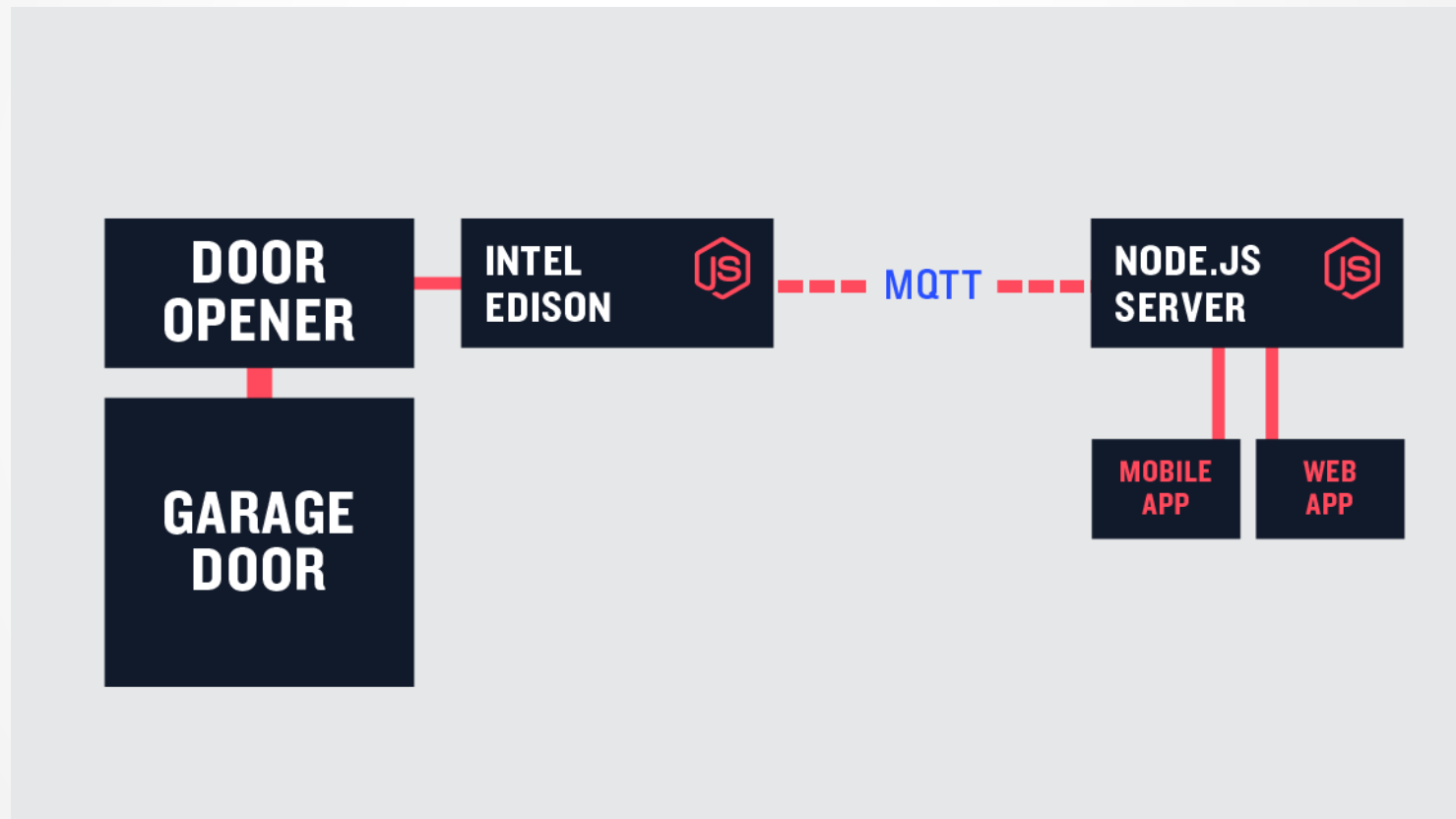
Diagrama de Comunicação



Diagrama de Comunicação



Exemplo de Aplicação



Características

- Quais são as portas padrão para uso com o MQTT?
 - A porta TCP/IP 1883 é reservada pela IANA para uso com MQTT. A porta TCP/IP 8883 também está registrada, para usar o MQTT sobre SSL.
- Qual o suporte que o MQTT tem para segurança dos dados?
 - Você pode passar um nome de usuário e uma senha com um pacote MQTT na versão 3.1 do protocolo. A criptografia em toda a rede pode ser tratada com SSL, independentemente do protocolo MQTT em si (vale a pena notar que o SSL não é o mais leve dos protocolos, e acrescenta importante sobrecarga de rede). Segurança adicional pode ser adicionada por um aplicativo de criptografia de dados que envia e recebe, mas isso não é algo embutido no protocolo, a fim de mantê-lo simples e leve.

Terminologia

- Conexão de rede
 - Uma construção fornecida pelo protocolo de transporte subjacente que está sendo usado pelo MQTT.
 - Ele conecta o Cliente ao Servidor.
 - Ela fornece os meios para enviar um fluxo de bytes ordenado, sem perdas, em ambas as direções.
- Mensagem da Aplicação:
 - Os dados transportados pelo protocolo MQTT através da rede para a aplicação. Quando as mensagens do aplicativo são transportadas pelo MQTT, elas têm uma qualidade de serviço e um nome do tópico associados.

Terminologia

- Cliente:
 - Um programa ou dispositivo que usa MQTT. Um cliente sempre estabelece a conexão de rede com o servidor.
 - Ele pode:
 - Publicar mensagens de aplicativos em que outros clientes possam estar interessados.
 - Inscrever-se para solicitar as mensagens de aplicação que está interessado em receber.
 - Anular a subscrição para remover uma solicitação de mensagens do aplicativo.
 - Desconectar-se do servidor.

Terminologia

- Servidor:
 - Um programa ou dispositivo que atua como intermediário entre Clientes que publicam Mensagens do Aplicativo e Clientes que fizeram Assinaturas.
 - Um Servidor:
 - Aceita conexões de rede de clientes.
 - Aceita mensagens de aplicativos publicadas pelos clientes.
 - Processa solicitações de Inscrição e Cancelamento de clientes.
 - Encaminha mensagens de aplicativos que correspondem a assinaturas de clientes.

Terminologia

- Subscrição:
 - Uma Assinatura compreende um Filtro de Tópicos e um QoS máximo. Uma Assinatura está associada a uma única Sessão. Uma Sessão pode conter mais de uma Assinatura. Cada Assinatura dentro de uma sessão tem um Filtro de Tópico diferente.
- Nome de Tópico:
 - O rótulo anexado a uma mensagem de aplicativo correspondente às assinaturas conhecidas pelo servidor. O Servidor envia uma cópia da Mensagem do Aplicativo para cada Cliente que tenha uma Assinatura correspondente.

Terminologia

- Filtro de Tópico:
 - Uma expressão contida em uma Assinatura, para indicar interesse em um ou mais tópicos. Um filtro de tópico pode incluir caracteres curinga.
- Sessão:
 - Uma interação com estado entre um cliente e um servidor. Algumas Sessões duram apenas enquanto dura a Conexão de Rede, outras podem abranger várias Conexões de Rede consecutivas entre um Cliente e um Servidor.
- Pacote de Controle MQTT:
 - Um pacote de informação que é enviado através da rede de interconexão. A especificação MQTT define quatorze tipos diferentes de pacotes de controle, um dos quais (o pacote PUBLISH) é utilizado para realizar mensagens de aplicações.

Conceitos Básicos de Funcionamento

- Publicar / Assinar
 - O protocolo MQTT baseia-se no princípio de publicar mensagens e subscrever tópicos, que é normalmente referido como um modelo de publicação / subscrição. Os clientes podem subscrever os tópicos que pertencem a eles e assim receber quaisquer mensagens que sejam publicadas para esses tópicos. Como alternativa, os clientes podem publicar mensagens para tópicos, tornando-os disponíveis para todos os assinantes desses tópicos.
- Tópicos e assinaturas
 - As mensagens no MQTT são publicadas em tópicos, que podem ser considerados como áreas de assunto. Os clientes, por sua vez, se inscrevem para receber mensagens específicas assinando um tópico. As assinaturas podem ser explícitas, o que limita as mensagens recebidas para o tópico específico em questão ou pode usar designadores curinga, como um sinal de número (#) para receber mensagens para uma variedade de tópicos relacionados.

Conceitos Básicos de Funcionamento

- Níveis de qualidade de serviço
 - O MQTT define três níveis de qualidade de serviço (QoS) para entrega de mensagens, com cada nível designando um nível mais alto de esforço pelo servidor para garantir que a mensagem seja entregue.
 - Níveis mais elevados de QoS garantem a entrega mais confiável de mensagens, mas podem consumir mais largura de banda da rede ou submeter a mensagem a atrasos devido a problemas como latência.
- Mensagens retidas
 - Com o MQTT, o servidor mantém a mensagem mesmo depois de enviá-la a todos os assinantes atuais. Se uma nova assinatura for enviada para o mesmo tópico, todas as mensagens retidas serão enviadas para o novo cliente assinante.

Conceitos Básicos de Funcionamento

- Sessões limpas e conexões duráveis
 - Quando um cliente MQTT se conecta ao servidor, ele define o sinalizador de sessão limpa. Se o sinalizador estiver definido como true, todas as assinaturas do cliente serão removidas quando ele se desconectar do servidor. Se o sinalizador estiver definido como false, a conexão será tratada como durável e as assinaturas do cliente permanecerão em vigor após qualquer desconexão. Neste caso, as mensagens subsequentes que chegam com uma designação QoS elevada são armazenadas para entrega depois que a ligação é restabelecida. Usar o sinalizador de sessão limpa é opcional.

Conceitos Básicos de Funcionamento

- Testamentos (Wills)
 - Quando um cliente se conecta a um servidor, ele pode informar ao servidor que ele tem um testamento, ou uma mensagem, que deve ser publicada para um tópico específico ou tópicos no caso de uma desconexão inesperada. Um testamento é particularmente útil em configurações de alarme ou de segurança onde os gerentes de sistema devem saber imediatamente quando um sensor remoto perdeu o contato com a rede.
- Tópicos, árvores e strings
 - A distribuição de mensagens em sistemas baseados em MQTT depende da designação de tópicos específicos, árvores de tópicos e sequências de tópicos.

Conceitos Básicos de Funcionamento

- Tópicos
 - Os clientes são responsáveis por classificar os assuntos da mensagem que publicam em tópicos. Um tópico define o conteúdo de uma mensagem ou a área de assunto sob a qual a mensagem pode ser categorizada.
 - Os tópicos são importantes porque, enquanto as mensagens em sistemas ponto-a-ponto são enviadas para endereços de destino específicos, as mensagens em sistemas de publicação / subscrição, como o MQTT, são distribuídas com base na escolha de tópicos de cada assinante.
 - Ao se inscrever em um tópico específico, o assinante está se inscrevendo para receber todas as mensagens publicadas nesse tópico a partir de qualquer cliente emissor.

Conceitos Básicos de Funcionamento

- Árvóres de Tópicos
 - Normalmente, os tópicos são organizados hierarquicamente em árvores de tópicos, usando o caractere '/' para criar subtópicos na sequência de tópicos. Um exemplo de uma árvore de tópico simples poderia ser caminhão/carga.
- Cadeias (Strings) de Tópicos
 - Uma cadeia de tópicos é uma sequência de caracteres que identifica o tópico de uma mensagem de publicação/inscrição. As sequências de tópicos podem conter um dos dois esquemas de curinga para permitir que os assinantes combinem padrões dentro das cadeias definidas nas mensagens dos publicadores:
 - Multilevel: O caractere curinga (#) é usado para coincidir com qualquer número de níveis dentro de um tópico. Por exemplo, os assinantes do caminhão/carga/# recebem todas as mensagens que são designadas para os tópicos caminhão/carga e caminhão/carga/rfid.
 - Único nível: O caractere curinga (+) é usado para corresponder apenas a um nível de tópico. Por exemplo, caminhão /+ corresponde a caminhão/carga mas não a caminhão/carga/rfid.

MQTT Brokers

- WebSphere MQ
 - <http://www.ibm.com/developerworks/downloads/ws/wmq/>
- Really Small Message Broker and MQ Telemetry daemon for devices
 - https://www14.software.ibm.com/webapp/iwm/web/reg/download.do?source=AW-0U9&S_PKG=0U9
- Mosquitto
 - <http://mosquitto.org/>
- Micro broker in IBM Lotus Expeditor integrator
 - <http://www.ibm.com/developerworks/downloads/lx/lxpd/>

Clientes MQTT (Depuração)

- mqtt-spy (Windows, Linux, Mac)
 - <https://kamilfb.github.io/mqtt-spy/>
- MQTT.fx (Mac OSX, Windows and Linux)
 - <http://www.mqttfx.org/>
- MQTTInspector (iOS)
 - <https://github.com/ckrey/MQTTInspector>
- MQTTLens (Windows, Linux & MacOSX)
 - <https://github.com/sandro-k/mqtt-lens>
- MQTTBox(Windows, Linux & MacOSX)
 - <http://workswithweb.com/mqttbox.html>

MQTT.js

MQTT.js

- MQTT.js é uma biblioteca cliente para o protocolo MQTT, escrito em JavaScript tanto para o node.js como para o navegador.
- Serve para os clientes tanto publicarem como assinarem os tópicos.
- Disponível em <https://www.npmjs.com/package/mqtt>
- Muitas aplicações do MQTT podem ser implementadas usando apenas as mensagens CONNECT, PUBLISH, SUBSCRIBE, e DISCONNECT.
- Instalação;

```
$ npm install mqtt --save [-g]
```

MQTT.js

```
var mqtt = require('mqtt');  
var client = mqtt.connect('mqtt://test.mosquitto.org');  
client.on('connect', function () {  
  console.log("Conectado ao Broker");  
  client.subscribe('presence');  
  client.publish('presence', 'Alo, mensagem via mqtt');  
})  
client.on('message', function (topic, message) {  
  // message é do tipo Buffer  
  console.log(message.toString());  
  client.end();  
})
```

MQTT.js

- MQTT.js agrupa um comando para interagir com um broker. Para tê-lo disponível em seu caminho, você deve instalar MQTT.js globalmente:

```
$ npm install mqtt -g
```

- Então, digite em um terminal

```
$ mqtt sub -t 'hello' -h 'test.mosquitto.org' -v
```

- Em outro terminal digite:

```
$ mqtt pub -t 'hello' -h 'test.mosquitto.org' -m 'from MQTT.js'
```

MQTT.js

- `mqtt.connect([url], options)`
- Conecta ao broker especificado pela URL fornecida e retorna um Client.
- A URL pode ser algum dos seguintes protocolos: 'mqtt', 'mqttps', 'tcp', 'tls', 'ws', 'wss'. A URL pode ser um objeto retornado pela função `URL.parse()`, neste caso os dois objetos são mesclados, isto é, você pode passar um único objeto com a URL e as opções de conexão.
- Você também pode especificar as opções de servidor com o seguinte conteúdo: `[{ host: 'localhost', port: 1883 }, ... ]`, neste caso o vetor é iterado a cada conexão.
- Para todas as opções relacionadas ao protocolo MQTT, veja o construtor Client.

MQTT.js

- `mqtt.Client(streamBuilder, options)`
- A classe Client envelopa um conexão cliente a um broker MQTT sobre um método de transporte escolhido (TCP, TLS, WebSocket, etc).
- Client automaticamente cuida dos seguintes eventos:
 - Pedidos regulares de ping do servidor
 - Fluxo QoS
 - Reconexões automáticas
 - Inicia a publicação antes de ser conectado

Eventos

- Evento 'connect' function (connack) {}
 - Emitido em uma (re)conexão bem sucedida (i.e. connack rc=0).
 - connack: pacote connack recebido. Quando a opção de conexão clean é falsa e o servidor tem uma sessão anterior para o mesmo cliente, a flag connack.sessionPresent é verdadeiro. Quando este for o caso, você deve se basear na sessão armazenada e preferir não enviar comando de subscrição para o cliente.
- Evento 'reconnect' function () {}
 - Emitido quando uma reconexão é iniciada.
- Evento 'close' function () {}
 - Emitido depois de uma desconexão.
- Evento 'offline' function () {}
 - Emitido quando um cliente fica “offline”.
- Evento 'error' function (error) {}
 - Emitido quando o cliente não consegue se conectar (i.e. connack rc != 0) ou ocorre um erro com os parâmetros passados.

Sobre QoS

- Aqui descrevemos como a QoS funciona:
 - QoS 0: Recebido no máximo uma vez: o pacote é enviado, e é isso. Não há validação sobre se ele foi recebido ou não.
 - QoS 1: Recebido pelo menos uma vez: O pacote é enviado e armazenado desde que o cliente não tenha recebido uma confirmação do servidor. MQTT garante que ele será recebido, mas pode haver duplicatas.
 - QoS 2: Recebido exatamente uma vez: O mesmo que QoS 1, mas não há duplicatas.
- Sobre o consumo de dados, evidentemente, $QoS\ 2 > QoS\ 1 > QoS\ 0$, se isso for uma preocupação para você.

MQTT.js

- `mqtt.Client#publish(topic, message, [options], [callback])`
- Publica uma mensagem em um tópico
 - topic é o tópico a ser publicado, String
 - message é a mensagem a ser publicada, Buffer or String
 - options são as opções a serem publicadas, incluindo:
 - Qos: QoS level, Number, default 0
 - Retain: retain flag, Boolean, default false
 - Dup: mark as duplicate flag, Boolean, default false
 - callback - function (err), disparada quando a manipulação de QoS termina, ou no próximo tick se QoS 0. Um erro ocorre se o cliente está desconectando.

Eventos

- Evento 'message'

function (topic, message, packet) {}

- Emitido quando o cliente recebe um pacote publicado

- topic: tópico do pacote recebido
- message: conteúdo do pacote recebido
- packet: o pacote recebido, conforme definido em mqtt-packet

- Event 'packetsend'

function (packet) {}

- Emitido quando o cliente envia um pacote. Isso inclui pacotes .published() assim como pacotes usados pelo MQTT para gerenciar assinaturas e conexões.

- packet: o pacote recebido, conforme definido em mqtt-packet

MQTT.js

- `mqtt.Client#subscribe(topic/topic array/topic object, [options], [callback])`
- Assinatura de um ou mais tópicos
 - Topic: é uma String tópico para se inscrever ou um array de tópicos para se inscrever. Ele também pode ser um objeto, que tem como chaves do objeto o nome do tópico e o valor de QoS, como `{'test1': 0, 'test2': 1}`. Os caracteres coringa para os tópicos MQTT são suportados (+ - para um único nível e # - para multinível)
 - Options: são as opções para a assinatura, incluindo:
 - Qos, o nível QoS de assinatura, default 0
 - callback - function (err, granted) callback disparado em suback onde:
 - Err: um erro de assinatura ou um erro que ocorre quando o cliente está desconectando
 - Granted: é um array de {topic, qos} onde:
 - Topic: é um tópico que é subscrito
 - Qos: é o nível QoS garantido para a assinatura

MQTT.js

- `mqtt.Client#unsubscribe(topic/topic array, [callback])`
- Desinscreve de um ou mais tópicos
 - Topic: é uma String com o tópico ou array de tópicos para se desinscrever.
 - callback - function (err), disparada em unsuback. Um erro acontece se o cliente está desconectando.

Exemplo

```
var mqtt = require('mqtt'), url = require('url');  
// Parse  
var mqtt_url = url.parse(process.env.CLOUDMQTT_URL || 'mqtt://localhost:1883');  
var auth = (mqtt_url.auth || ':').split(':');  
var url = "mqtt://" + mqtt_url.host;  
var options = {  
  port: mqtt_url.port,  
  clientId: 'mqttjs_' + Math.random().toString(16).substr(2, 8),  
  username: auth[0],  
  password: auth[1],  
};  
// Create a client connection  
var client = mqtt.connect(url, options);
```

Exemplo

```
client.on('connect', function() { // When connected
  // subscribe to a topic
  client.subscribe('hello/world', function() {
    // when a message arrives, do something with it
    client.on('message', function(topic, message, packet) {
      console.log("Received " + message + " on " + topic + "");
    });
  });
  // publish a message to a topic
  client.publish('hello/world', 'my message', function() {
    console.log("Message is published");
    client.end(); // Close the connection when published
  });
});
```


Apresentação

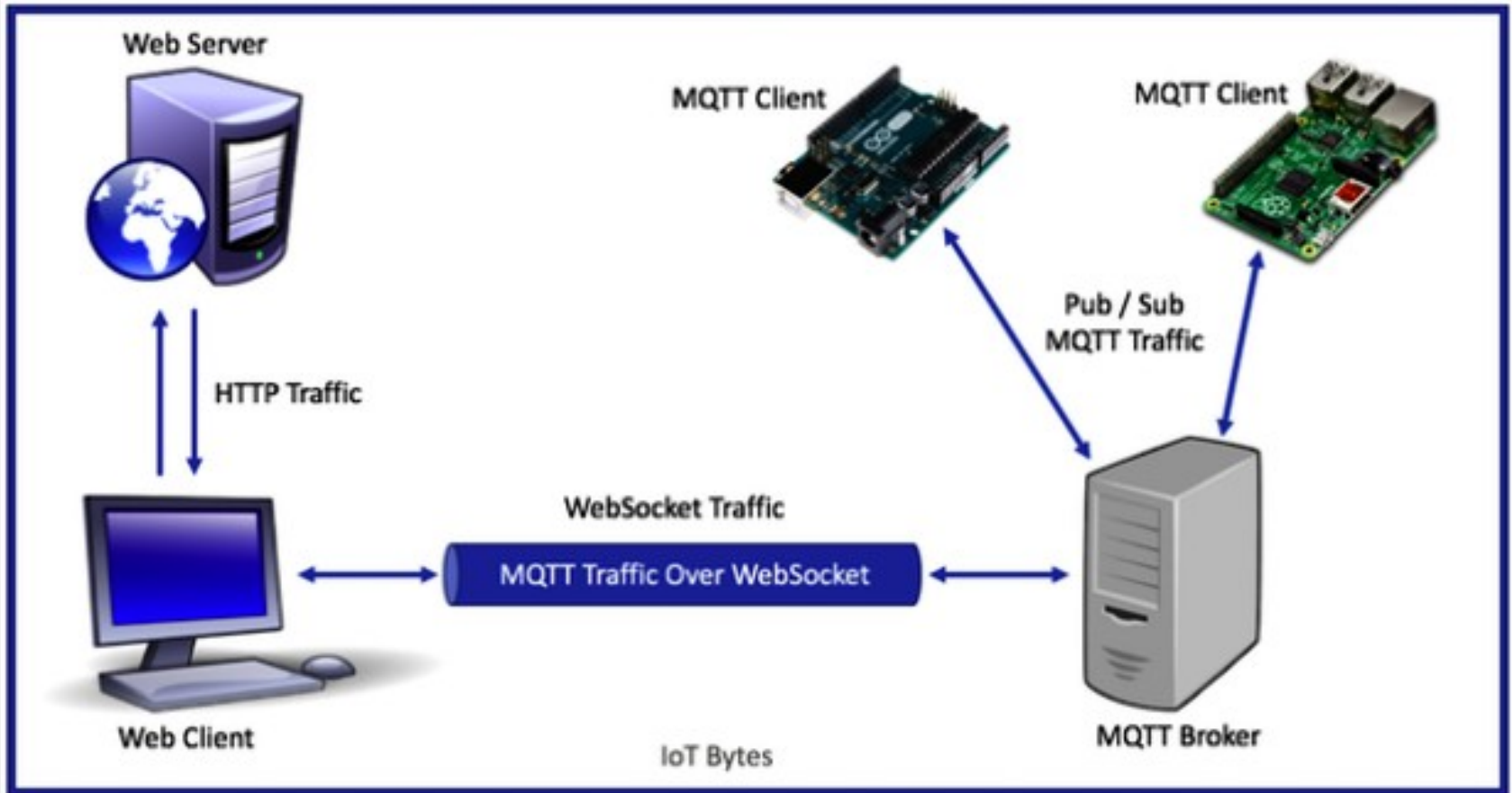
- <http://www.hivemq.com/blog/build-javascript-mqtt-web-application>
- <http://www.hivemq.com/blog/mqtt-essentials-part2-publish-subscribe>
- <https://blog.risingstack.com/getting-started-with-nodejs-and-mqtt/>
- <https://mosquitto.org/>
- <http://www.hivemq.com/blog/how-to-get-started-with-mqtt>
-

MQTT Over Websockets

MQTT sobre WebSockets

- O MQTT é ideal para uso em dispositivos com poucos recursos e para redes não confiáveis. É também perfeito para o envio de mensagens com baixo “overhead”.
- Seria bom enviar e receber mensagens MQTT diretamente no navegador de um celular ou desktop. Isso é possível com o uso de MQTT sobre WebSockets. Isso permite ao navegador fazer uso de todas as facilidades do MQTT e pode ser utilizado nos seguintes exemplos:
 - Mostrar a informação em tempo real de um dispositivo ou sensor;
 - Receber notificações “push”, por exemplo, se ocorrer um alerta ou uma condição crítica;
 - Comunicação eficiente com uma aplicação Web móvel.

MQTT sobre WebSockets



Browserify x Webpack

- O segredo para realizar isso, sem uso de novos módulos, é empacotar o MQTT.js para uso direto pelo navegador.
- Para isso deve-se utilizar as ferramentas browserify e webpack.
- O browserify é uma ferramenta que permite a utilização no navegador de módulos desenvolvidos para o nodejs.
- Basicamente ela verifica todas as dependências que o módulo necessita e junta todos os módulos em um único pacote (bundler) de modo que possa ser utilizado pelo navegador sem recorrer a dependências externas.
- O webpack é um outro tipo de “blunder” com uma sintaxe e características um pouco diferente do browserify,

Browserify

- De modo a usar o MQTT.js como um módulo browserify você pode utilizar os seguintes comandos:

```
$ npm install -g browserify // instala o browserify
```

```
$ cd node_modules/mqtt
```

```
$ npm install . // instala as dependencias de desenvolvimento
```

```
$ browserify mqtt.js -s mqtt > browserMqtt.js // requer o mqtt no seu cliente
```

Webpack

- Assim como o browserify o webpack exporta o MQTT.js como uma biblioteca. Para isso faça uso dos seguintes comandos:

```
$ npm install -g webpack // instala o webpack
```

```
$ cd node_modules/mqtt
```

```
$ npm install . // instala as dependencias de desenvolvimento
```

```
$ webpack mqtt.js ./browserMqtt.js --output-library mqtt
```

Websockets

```
<html> <head>
  <title>test Ws mqtt.js</title>
</head>
<body>
<script src="./browserMqtt.js"></script>
<script>
  var client = mqtt.connect() // you add a ws:// url here
  client.subscribe("mqtt/demo")
  client.on("message", function (topic, payload) {
    alert([topic, payload].join(": "))
    client.end()
  })
  client.publish("mqtt/demo", "hello world!")
</script> </body> </html>
```


Websockets

```
<html> <head>
  <script src="/mqtt.js"></script>
</head>
<body>
  <script>
    var client = mqtt.connect();
    client.subscribe("mqtt/demo");
    client.on("message", function(topic, payload) {
      alert([topic, payload].join(": "));
      client.end();
    });
    client.publish("mqtt/demo", "hello world!");
  </script>
</body> </html>
```